UNIPÊ
Centro Universitário
de João Pessoa

DESENVOLVIMENTO FRONT-END PARA WEB



git flow

CIÊNCIA DA COMPUTAÇÃO

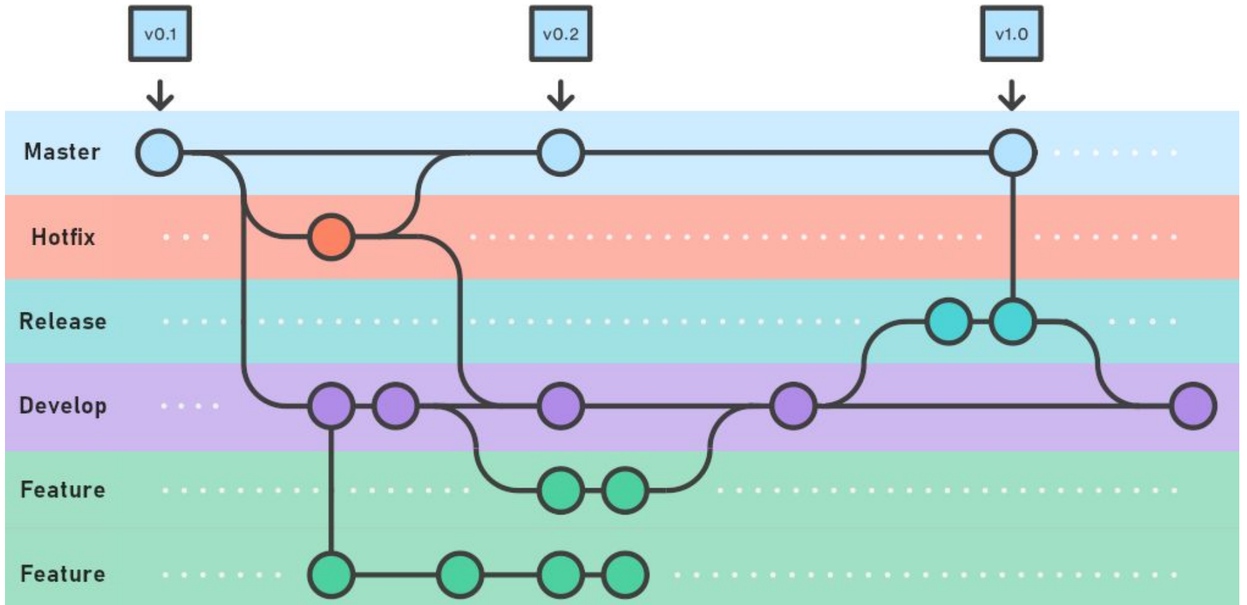
CIÊNCIA DA COMPUTAÇÃO

Prof. Israel Cunha

Git!

São conceitos fundamentais de controle de versão e colaboração em desenvolvimento de software.

Nos próximos slides estão alguns dos comandos essenciais que você deve cobrir:



Gitflow — Modelo de Branches

O Gitflow define um padrão de branches para organizar o desenvolvimento:

🟢 **main/master**
Código em produção. Sempre estável. Nunca commit direto aqui.

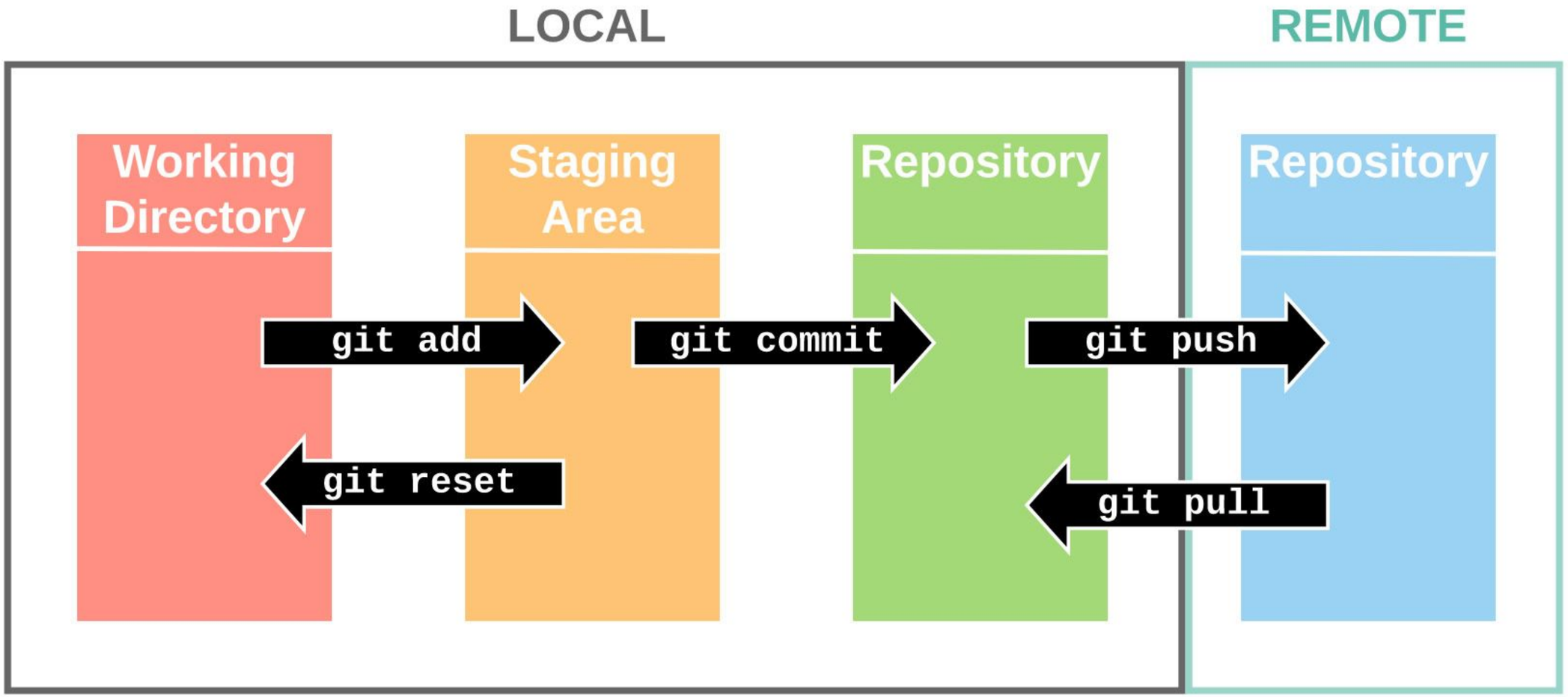
🔵 **develop**
Branch de integração. Reúne as features antes de ir para main.

🟣 **feature/***
Uma branch por funcionalidade. Ex: feature/login, feature/navbar

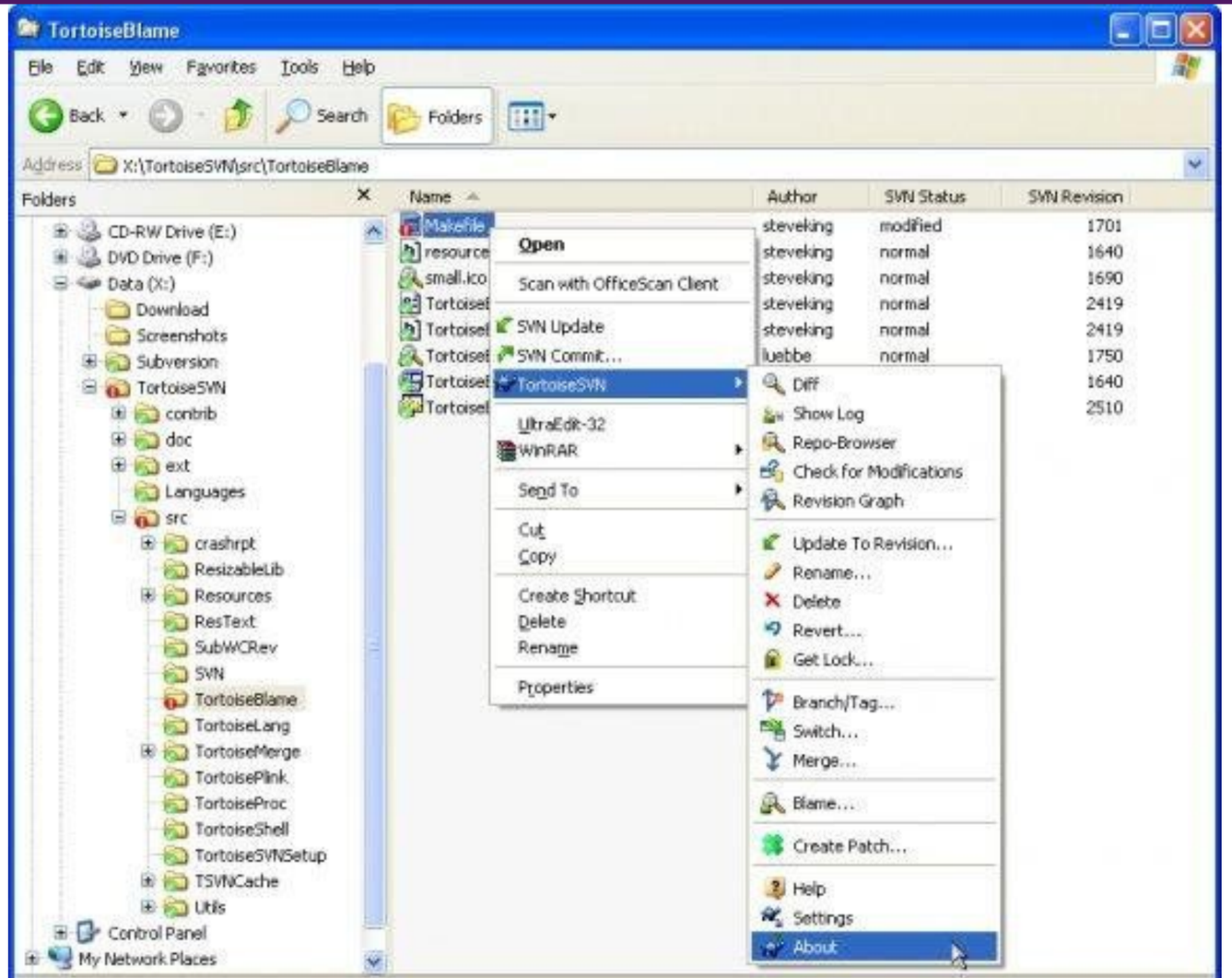
🟠 **release/***
Preparação de uma nova versão. Pequenos ajustes antes do deploy.

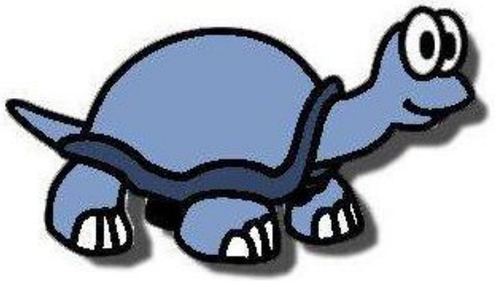
🔴 **hotfix/***
Correção urgente em produção. Parte da main, volta pra main e develop.

Comandos Git!



O Problema, antes do Git





TortoiseSVN

O Problema, antes do Git

Você já passou por isso?

- 📁 trabalho.zip
- 📁 trabalho_v2.zip
- 📁 trabalho_final.zip
- 📁 trabalho_final_AGORA.zip
- 📁 **trabalho_final_AGORA_DE_VERDADE.zip**

Git resolve isso.

Ele guarda o histórico completo de todas as mudanças, permitindo voltar a qualquer ponto no tempo — sem precisar de .zip.

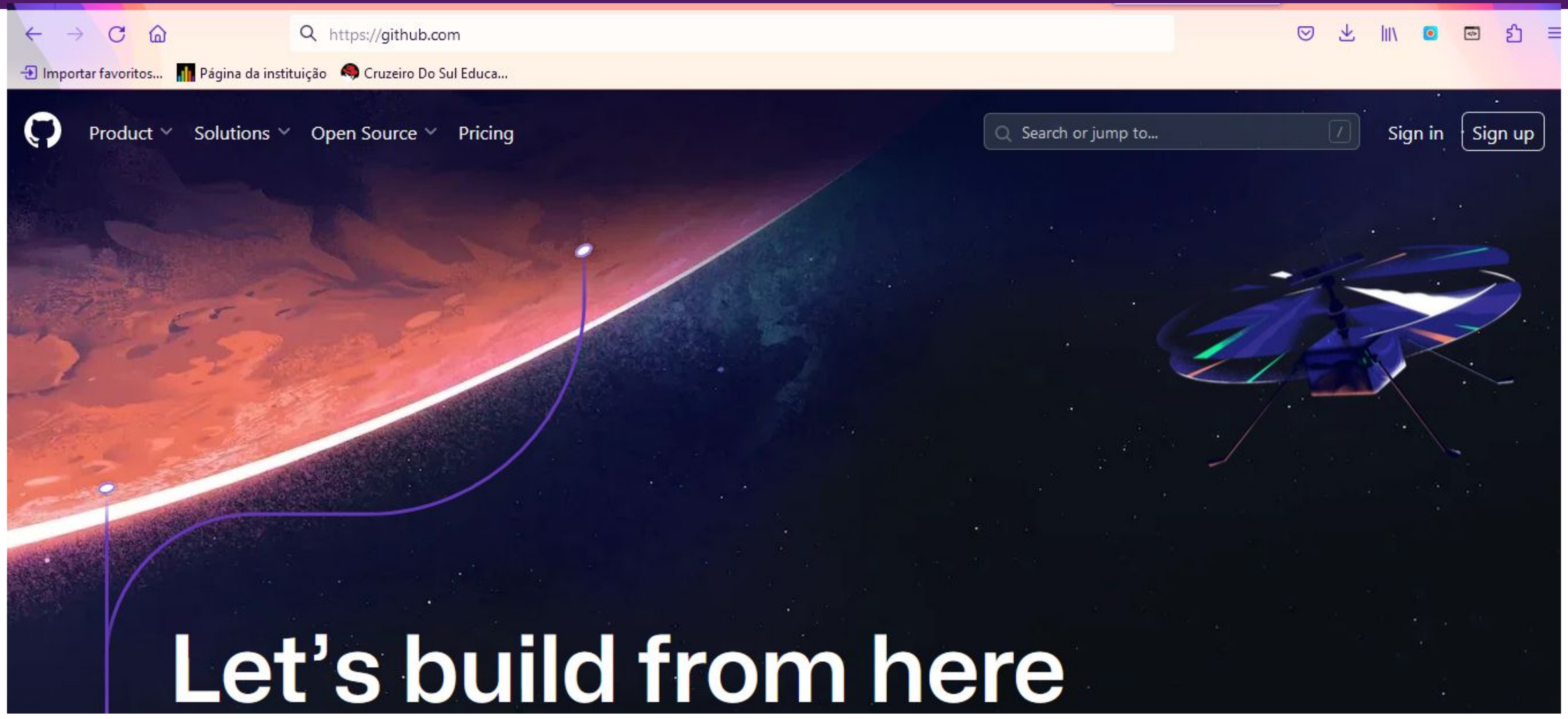
Git Bash



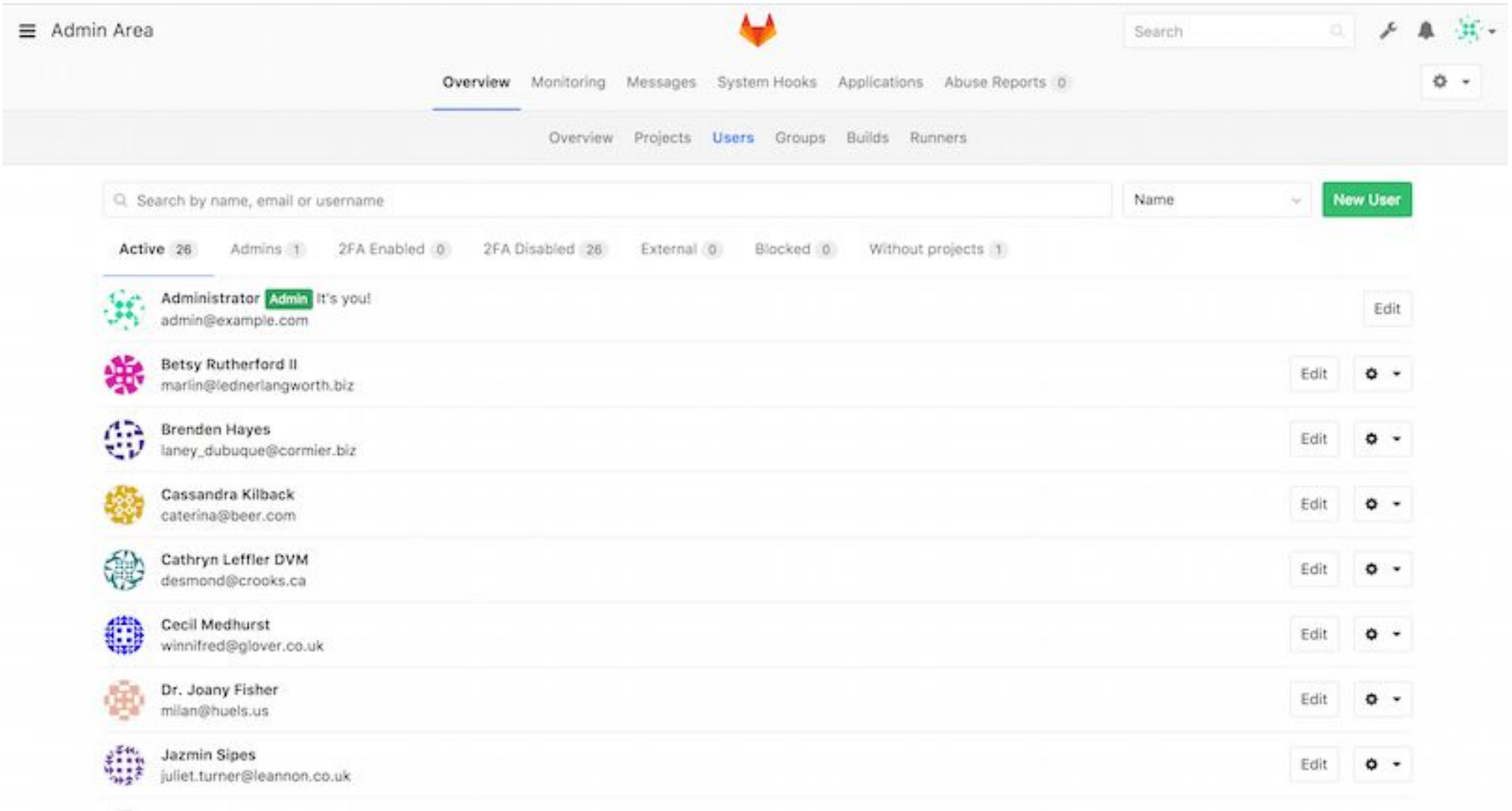
Install Git Bash on Windows

LINK

GitHub


https://github.com
Importar favoritos...
Página da instituição
Cruzeiro Do Sul Educa...
Product
Solutions
Open Source
Pricing
Search or jump to...
Sign in
Sign up
Let’s build from here

GitLab


Admin Area
Search
Overview
Monitoring
Messages
System Hooks
Applications
Abuse Reports 0
Overview
Projects
Users
Groups
Builds
Runners
Search by name, email or username
Name
New User
Active 26
Admins 1
2FA Enabled 0
2FA Disabled 26
External 0
Blocked 0
Without projects 1
Administrator Admin It's you!
admin@example.com
Edit
Betsy Rutherford II
marlin@lednerlangworth.biz
Edit
Brenden Hayes
laney_dubuque@cormier.biz
Edit
Cassandra Kilback
caterina@beer.com
Edit
Cathryn Leffler DVM
desmond@crooks.ca
Edit
Cecil Medhurst
winnifred@glover.co.uk
Edit
Dr. Joany Fisher
milan@huels.us
Edit
Jazmin Sipes
juliet.turner@leannon.co.uk
Edit

Comandos git!

Set username:

git config --global user.name "FIRST_NAME LAST_NAME"

Set seu endereço de email:

git config --global user.email "MY_NAME@example.com”

Comandos git!

```
echo "# aula" >> README.md

git init

git add README.md

git commit -m "first commit"

git branch -M main

git remote add origin https://github.com/israelcunha/aula.git

git push -u origin main
```

Comandos git!

git remote add origin https://github.com/israelcunha/aula.git

git branch -M main

git push -u origin main

Comandos git!

git init: Inicializa um novo repositório Git em um diretório.

git init NovoDiretorio

Comandos git!

git clone: Clonar um repositório existente para o seu ambiente local.

git clone ssh

git clone html

Comandos git!

git add: Adicionar arquivos ao índice (staging area) para prepará-los para o commit.

git add file

git add .

Comandos git!

git commit: Cria um novo commit com as alterações adicionadas no índice.

git commit -m “Novo Commit”

git commit -m “Segundo commit” path/to/file.xpto

Comandos git!

git status: Exibe o status atual dos arquivos no repositório, incluindo quais estão no índice, quais foram modificados e quais estão ignorados.

git status

Comandos git!

git log: Mostra o histórico de commits do repositório.

git log

Comandos git!

gitk : Ele proporciona uma interface gráfica do usuário que ajuda na visualização da mecânica interna.

gitk

Comandos git!

git pull: Puxa as alterações mais recentes de um repositório remoto para o seu repositório local.

git pull

git pull origin main

Comandos git!

git push: Envia os commits locais para um repositório remoto.

git push main

git push origin main

Comandos git!

git branch: Lista, cria ou exclui branches (ramificações).

git branch

git branch -d nome-da-branch

git checkout -b nome-da-branch

Comandos git!

git checkout: Alterna entre branches e permite criar novos branches.

git checkout -b nome-da-branch

git checkout nome-da-branch

git checkout .

Comandos git!

git merge: Combina alterações de uma branch com outra.

git merge branch-nova

Comandos git!

git fetch: Busca as alterações mais recentes dos repositórios remotos, mas não as aplica.

git fetch

Comandos git!

git stash: Armazena temporariamente as alterações não comitadas para que você possa mudar de branch sem commitar.

git stash

Comandos git!

git stash pop: O pop vai remover o stash mais recente da pilha e aplicar as mudanças que ele contém.

git stash pop

Comandos git!

git reset: Desfaz alterações em commits, índice ou diretório de trabalho.

git reset - - hard HEAD

Git vs GitHub vs GitLab

🔧 Git

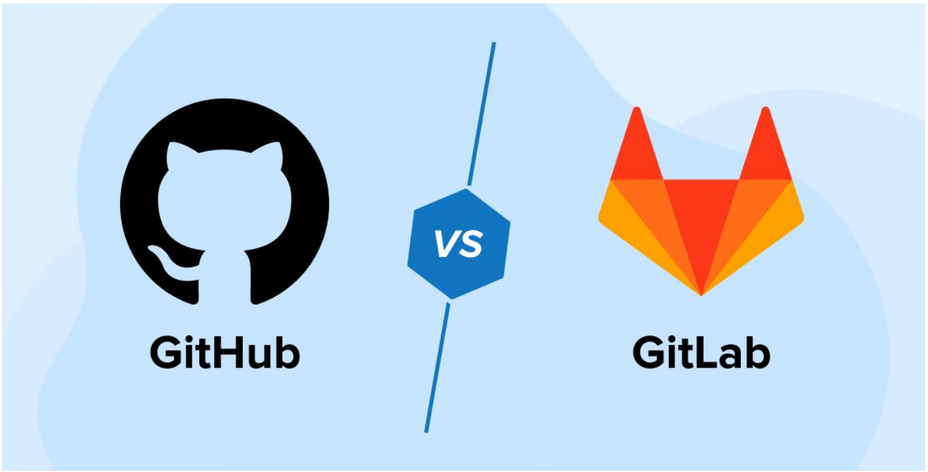
Ferramenta de controle de versão que roda na sua máquina.
Não precisa de internet. Funciona 100% local.

🐙 GitHub

Plataforma online para hospedar repositórios Git.
Foco em open source, portfólio e colaboração pública.
Pertence à Microsoft desde 2018.

🦊 GitLab

Plataforma similar ao GitHub, muito usada em empresas.
CI/CD integrado, self-hosted e foco em DevOps.



GitHub na Prática

O que você encontra no GitHub:

- ⭐ **Repositórios públicos e privados**
- 📋 **Issues — registrar bugs e tarefas**
- 🔀 **Pull Requests — propor alterações ao projeto**
- 📊 **Insights — Visualizar contribuições e histórico**
- 🚀 **GitHub Pages — hospedar sites estáticos de graça**



👉 **github.com — crie seu perfil, ele é seu portfólio profissional!**

GitLab na Prática

O que diferencia o GitLab:



- 🏢 **Muito usado em ambientes corporativos**
- 🔒 **Self-hosted: a empresa roda o GitLab nos próprios servidores**
- ⚙️ **CI/CD nativo — pipelines de build, teste e deploy**
- 🔀 **Merge Requests (equivalente ao Pull Request do GitHub)**

GitHub = portfólio e open source
GitLab = empresas e DevOps
(mas ambos fazem as duas coisas — isso é uma generalização útil)

.gitignore — O que não deve ir pro repositório

O arquivo .gitignore diz ao Git quais arquivos ignorar:

```
node_modules/     # dependências do projeto (podem ter +100k arquivos!)
.env              # variáveis de ambiente e senhas
dist/             # build gerada automaticamente
.DS_Store         # arquivo do macOS
*.log             # arquivos de log
```

Dica: github.com/github/gitignore tem templates prontos para cada linguagem.

Conflitos de Merge — Não entre em pânico!

O que é um conflito?

Acontece quando duas branches alteram a mesma linha do mesmo arquivo.
O Git não sabe qual versão manter — e pede sua decisão.



Como aparece no arquivo:

```
<<<<<<< HEAD
Seu código aqui
=======
Código da outra branch
>>>>>>> feature/x
```

Como resolver:

1. Edite o arquivo e escolha qual código manter (ou combine os dois)
2. Remover as marcações <<<<, ====, >>>>
3. git add . e git commit

Pull Request — Propondo Alterações

O que é um Pull Request (PR)?

É uma solicitação para mesclar sua branch ao projeto principal.
No GitLab é chamado de Merge Request (MR) — a ideia é a mesma.

Fluxo de um PR:

1. Cria uma branch: `git checkout -b feature/minha-feature`
2. Faz os commits: `git add .` → `git commit -m "feat: ..."`
3. Sobe para o remoto: `git push origin feature/minha-feature`
4. Abre o PR no GitHub/GitLab
5. Time revisa o código (code review)
6. PR aprovado → merge na main ou develop

Fluxo Completo — Da Feature ao Merge

Cenário: vou criar o botão de login do site

```
git checkout develop
git pull origin develop                    # garante que estou atualizado
git checkout -b feature/botao-login        # criar minha branch

# ... edito os arquivos HTML/CSS/JS ...

git add .
git commit -m "feat: adiciona botão de login"
git push origin feature/botao-login

# Abre o Pull Request no GitHub e aguarda aprovação ✅
```

Boas Práticas!

Além destes comandos, é importante também entender que existem os conceitos de branching, merging, pull requests (para ambientes de colaboração), resolução de conflitos e boas práticas no uso do Git.



Boas Práticas — Mensagens de Commit

Uma boa mensagem de commit explica, O QUE e POR QUÊ.

❌ Ruim:
git commit -m "fix"
git commit -m "alterações"
git commit -m "asdfgh"

✅ Bom (Conventional Commits):
git commit -m "feat: adiciona formulário de login"
git commit -m "fix: corrige validação do campo de email"
git commit -m "style: ajusta padding do header"

Prefixos comuns: feat | fix | style | refactor | docs

Boas Práticas — Mensagens de Commit

Estrutura e Padrões de Nome:

Use prefixos padrão: Categorize a ramificação pelo que ela faz (feature/, bugfix/, hotfix/, docs/, refactor/).

Adicione o identificador do chamado: Se usar ferramentas como Jira, coloque o código da tarefa logo após o prefixo (ex: feature/abc-456-...).

Separe com barras e hífens: Use barra (/) para separar o tipo do resto do nome e hífen (-) para separar os termos (kebab-case).

Seja direto: Crie nomes curtos que explique a mudança sem virar uma frase longa.

Boas Práticas — Branch Name

🚀 Novas Funcionalidades (`feature/`)

- `feature/PROJ-101-login-google`
- `feature/shopping-cart-page`
- `feature/user-profile-upload`

🐛 Correções de Erros Comuns (`bugfix/`)

- `bugfix/PROJ-204-checkout-crash`
- `bugfix/broken-footer-links`
- `bugfix/api-timeout-retry`

🚨 Correções Urgentes em Produção (`hotfix/`)

- `hotfix/PROJ-emergency-payment-failure`
- `hotfix/security-leak-patch`
- `hotfix/auth-token-expiration`

Boas Práticas — Branch Name

🔧 Refatoração e Melhoria de Código (`refactor/`)

- `refactor/PROJ-305-clean-database-queries`
- `refactor/optimize-image-loading`
- `refactor/simplify-auth-middleware`

📝 Documentação e Testes (`docs/` ou `test/`)

- `docs/update-readme-install`
- `docs/api-endpoints-swagger`
- `test/PROJ-402-add-unit-tests-login`

Boas Práticas — Criar Documentações

Estrutura Padrão Recomendada (README.md)

Título principal: Use apenas um # no topo com o nome claro do projeto.

Descrição curta: Explique o objetivo do software em no máximo três linhas.

Pré-requisitos: Liste as ferramentas, linguagens e versões necessárias para rodar o código.

Passo a passo de instalação: Insira os comandos exatos de terminal dentro de blocos de código.

Exemplos de uso: Demonstre como executar ou interagir com a aplicação.

Boas Práticas — Criar Documentações

Sintaxe Essencial e Exemplos

1. Títulos e Subtítulos

Use a quantidade de # para definir a hierarquia (de 1 a 6).

markdown

```
# Nome do Projeto (H1)

## Instalação (H2)

### Configuração de Variáveis de Ambiente (H3)
```

Boas Práticas — Criar Documentações

2. Blocos de Código

Sempre especifique a linguagem após as três crases para ativar o realce de cores. [1]

markdown

```bash
npm install
npm run dev
```

```javascript
const name = "Git branches";
console.log(name);
```

Boas Práticas — Criar Documentações

3. Listas e Destaques

Organize informações em tópicos simples e use negrito para termos críticos.

markdown

* **Importante:** Sempre configure o arquivo `.env` antes do deploy.

* Requisitos de sistema:
  - Node.js `v18.0.0` ou superior.
  - Banco de dados PostgreSQL.

Boas Práticas — Criar Documentações

4. Links e Imagens

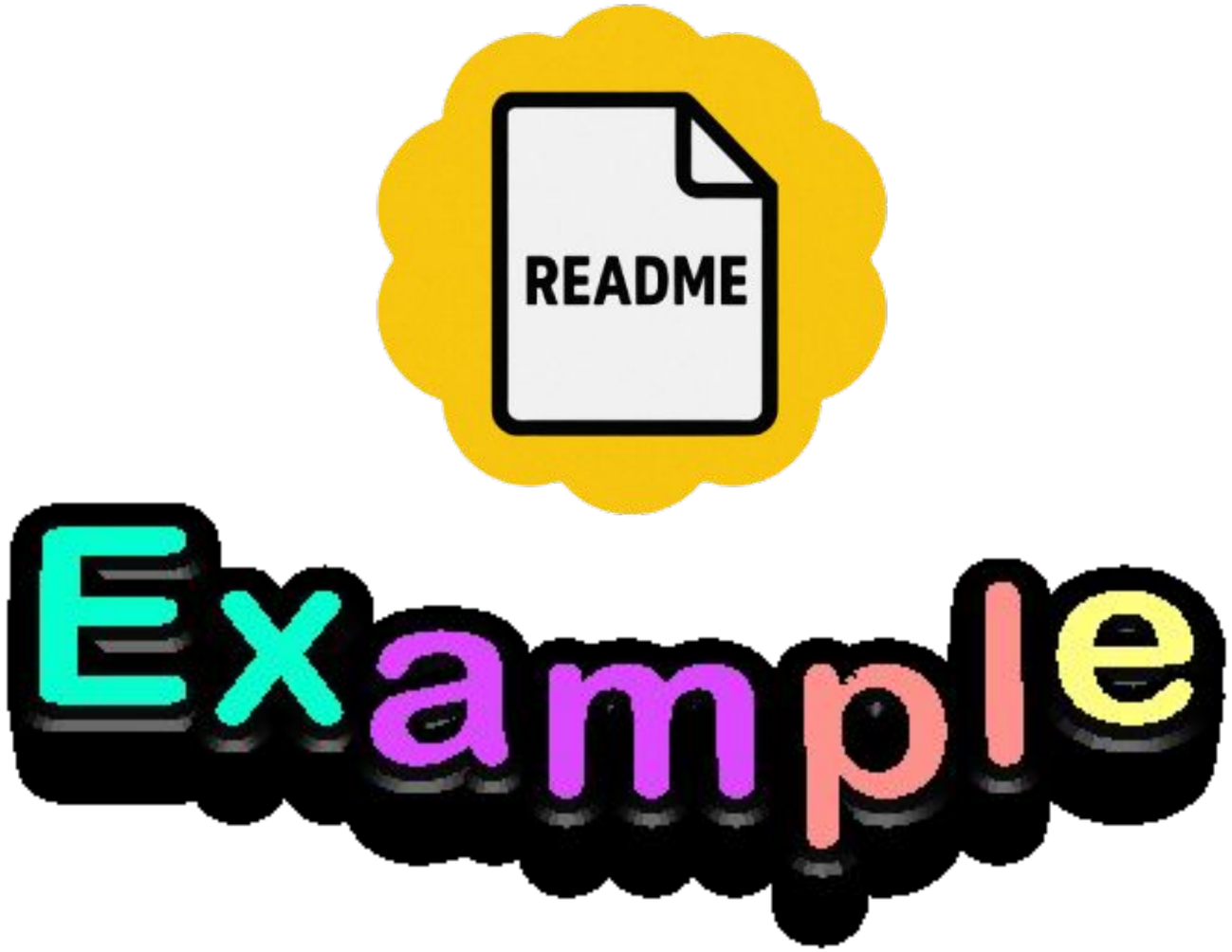
Mantenha os caminhos limpos, apontando para arquivos locais ou URLs externas.

markdown

[Leia o Guia de Contribuição](CONTRIBUTING.md)

![Logotipo do Projeto](assets/logo.png)

Boas Práticas — Criar Documentações



Boas Práticas — Criar Documentações

🌐 Nome do Seu Projeto

Uma breve descrição de duas ou três linhas explicando o objetivo principal deste projeto web e o problema que ele resolve.

🛠️ Pré-requisitos

Antes de começar, você vai precisar ter as seguintes ferramentas instaladas em sua máquina:

* **Navegador Web:** Google Chrome, Firefox, Edge ou Safari atualizados.

* **Editor de Código:** Visual Studio Code (recomendado) ou qualquer outro de sua preferência.

* **Extensão do VS Code (Opcional):** *Live Server* para rodar o projeto localmente com recarregamento automático.

Boas Práticas — Criar Documentações

🚀 Passo a passo de instalação

Siga as instruções abaixo para clonar o repositório e rodar o projeto localmente:

1. Clone este repositório para a sua máquina:

```bash
git clone https://github.com
```

Boas Práticas — Criar Documentações

2. Acesse a pasta do projeto:

```bash
cd nome-do-seu-repositorio
```

3. Abra o projeto no seu editor de código (caso use o VS Code):

```bash
code .
```

Boas Práticas — Criar Documentações

4. Abra o arquivo `index.html` diretamente no seu navegador ou clique em **Go Live** (se estiver usando a extensão Live Server).

💻 Exemplos de uso

O projeto interage diretamente com o usuário através da interface. Veja abaixo como utilizar as principais funções:

* **Preenchimento do Formulário:** Insira os dados solicitados nos campos de entrada da tela.
* **Execução da Ação:** Clique no botão principal para processar as informações via JavaScript.
* **Visualização do Resultado:** O resultado será renderizado na tela de forma responsiva graças ao CSS.

Boas Práticas — Criar Documentações

Exemplo de estrutura JavaScript utilizada para manipulação do DOM:

```javascript
// Captura do botão no HTML e execução do evento de clique
const mainButton = document.querySelector('#btn-action');

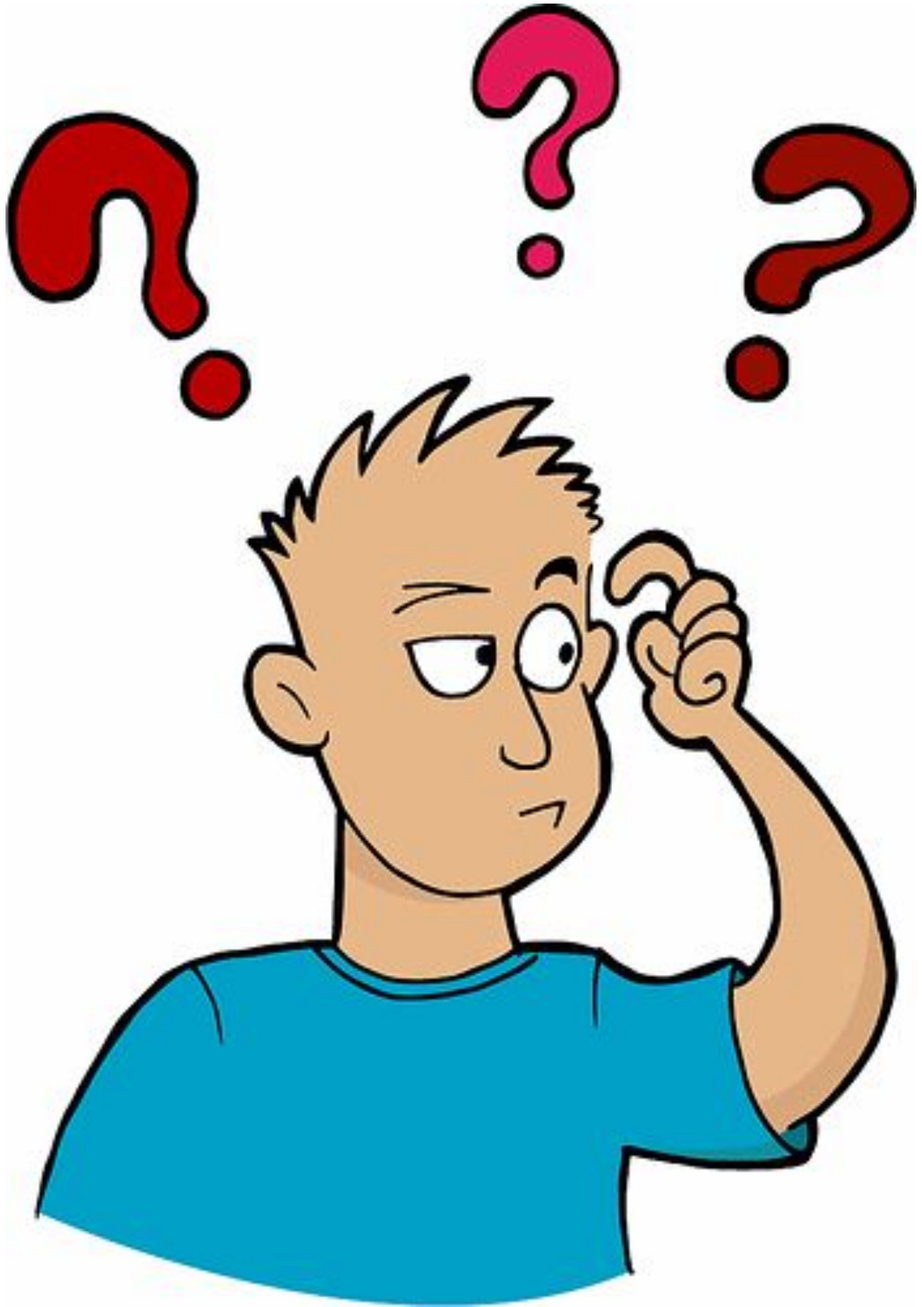

mainButton.addEventListener('click', () => {
    console.log('Ação executada com sucesso!');
});
```

Boas Práticas — Criar Documentações



As documentações dos seu repositório devem estar sempre atualizadas de acordo com o progresso do seu projeto que esta sendo desenvolvido ou em manutenção;

UNIPÊ
Centro Universitário
de João Pessoa



Atividade

Disponível no Blackboard

Criar uma conta no github , em seguida, subir um código simples com uma classe HelloWorld.html e um readme.md como documentação, descrevendo seu projeto inicial.

Enviar o link do repositório público no BB.



OBRIGADO!



B.Sc. SI - Israel Cunha
Pós-graduado Testes Ágeis
Cesar School - REC